Project Write-Up

CMPT 310, Group 1

github.com/praneetkb/2048-Master

1. System Explanation

1.1 What our system does

2048 is a sliding puzzle on a 4×4 grid. Every move pushes all the tiles in one direction, and when two tiles with the same number collide, they merge into one tile, adding the merged value to the score. After every move, the game adds a tile valued 2 or 4 on a random empty cell. The game ends when the board is full and no movement of tiles can happen.

Our system plays the game by itself: it takes the current board, returns one legal move, and repeats until no move changes the board. We wrote the engine ourselves, including tile movement and merging, score and rewards, the random 2 (90%) or 4 (10%) tile spawn, and the game-over check.

1.2 What AI methods we use

We use two methods from the course. The first is reinforcement learning: TD(0) over an N-tuple network, which learns a value function $V(s)$ from self-play. The second is Expectimax, the chance-node version of the adversarial search we covered in class, which uses that value function to look ahead through the random spawns.

1.3 The pipeline

We train the value function separately and save it to a file. During a game the agent only reads that file, so the two halves are independent and nothing is learned while playing.

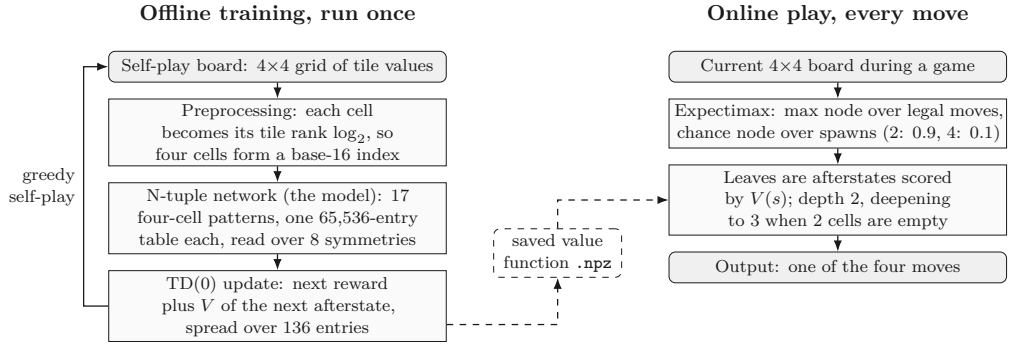

Figure 1. The two pipelines. Training writes the value function once, and play reads it on every move.

Both halves meet on afterstates, the boards you get after a move but before the spawn. A move in 2048 is deterministic and only the spawn is random, so the afterstate is the last position we can evaluate before luck enters. While training, we correct the value of the afterstate we just left using the reward of the next move plus the value of the next afterstate, or zero if the game ended there. The search stops at that same point, so the learned V goes straight into Expectimax with nothing to convert. The eight board symmetries are what make training affordable, since every position we see updates all 136 shared entries at once.

1.4 Results

We recorded the average score of every block of 1,000 self-play episodes, then played the finished agent against two baselines for 30 games each in the same environment.

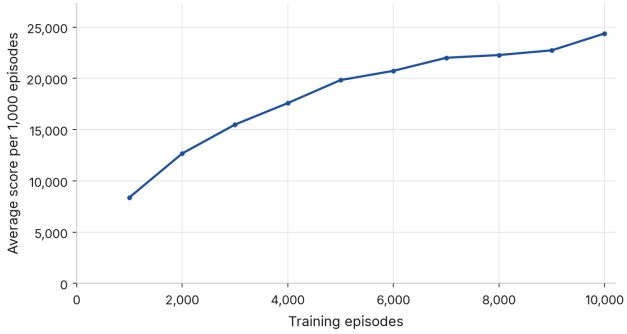

Figure 2. Average score per 1,000 training episodes. Self-play score rises from 8,384 to 24,382.

Agent	Games	Avg	Median	Best	Best tile	Reached 2048	ms/move
Random	30	1,178	990	3,076	256	0/30	0.0
Expectimax (heuristic)	30	14,724	15,698	27,308	2,048	5/30	8.8
Expectimax + RL	30	62,244	66,646	120,744	8,192	30/30	12.9

Table 1. The same search with a learned value function instead of a hand-written one, on the final code.

Swapping the hand-written evaluation for the learned one, inside the exact same search, multiplies the average score by 4.2 and takes the agent from 5 wins in 30 games to 30, at about 4 extra milliseconds per move.

1.5 Limitations and what we did about them

Three things still limit the agent. Learning starts slowly, because every TD update is divided among all 136 entries it touches, so the real step per weight is much smaller than α suggests; our first rate of 0.01 only reached about 15,500 after 10,000 episodes, and we settled on 0.05 after measuring both. Fixed depth 2 is also weakest exactly where games are lost, since on a nearly full board one bad spawn can leave no good move and a two-ply search sees only one spawn ahead. Two or fewer empty cells make the chance node small, so we deepen to 3 precisely there, which raises the cost from roughly 8 to 13 milliseconds per move and stays fast enough to watch. Finally, the value function knows nothing about board structure beyond its 17 local patterns, which is why we still need the search: playing greedily with no lookahead, the same network averages about 25,200 and wins half its games, against 62,244 and 30 out of 30 inside Expectimax. We kept the heuristic baseline for the same reason, to separate what learning contributes from what search does.

2. Feature Table

The table below summarizes the major components implemented for the 2048 environment, AI agents, reinforcement learning pipeline and user interface.

Description	Platform	Completeness	Code	Author(s)	Notes
4x4 board & tile representation	Local	5	Python (NumPy)	João	Represents the board, values for each tile and empty cells.
Tile movement and merging	Local	5	Python (NumPy)	Praneet	Implements up, down, left, and right movement and tile merging behavior when tiles collide.
Score, rewards, and game termination	Local	5	Python	Ryan	Handles rewards, score updates, legal-move checks, and game-over conditions.
Random tile spawning	Local	5	Python	Jayden	Randomly spawns a 2 or 4 tile on empty cells after a legal move using the probability distribution. Missing test cases.
Random agent	Local	5	Python	Jayden	Selects legal moves randomly and serves as a baseline for evaluating other stronger agents.
Heuristic Expectimax agent	Local	5	Python	João, Praneet, Ryan	Implements max and chance nodes, expected-value calculation, search depth, termination, and a hand-designed board evaluation function.
N-tuple network value function	Local	5	Python (NumPy)	João	Learns board-state values using multiple tuple patterns, lookup tables, and board symmetries.
TD learning training pipeline	Local	4	Python (NumPy)	Praneet	Trains the N-tuple value function through self-play using Temporal Difference learning and periodic checkpoints. Missing test cases.
RL and Expectimax integration	Local	5	Python	Ryan	Uses the learned N-tuple value function as the evaluation function inside the Expectimax search. Missing some test cases.
Agent evaluation and performance comparison	Local	5	Python	Jayden	Compares Random, heuristic Expectimax, and Expectimax + RL.
4x4 board and tile renderer	Local	5	Python (Pygame)	João	Displays the game board and tile values in the GUI.
Game loop and tile animations	Local	5	Python (Pygame)	Jayden	Connects and controls the overall gameplay and implemented tile animations for moving and merging.
Main menu and game header	Local	5	Python (Pygame)	Praneet, Ryan	Allows user to select between 3 agents to watch play and a comparison mode in the main menu. Displays the game header: title, score, best score, restart.
Automated testing	Local	3	Python (Pytest)	João, Ryan, Praneet	Tests cover core game mechanics, agents, N-tuple network behavior, and UI components. More tests could have been written.

3. External Tools & Libraries

Our project was implemented in Python and we used some external libraries to support numerical computation, reinforcement learning, visualization, testing, and the graphical user interface.

- Matplotlib: Used to generate the TD-learning training-progress graph showing average score across training episodes.
- Pytest: Used for automated testing of the game environment, agents, N-tuple network, and UI components.
- NumPy: Used for efficient representation and manipulation of the 4x4 game board, movement operations, and N-tuple network computations.
- Pygame: Used to implement the graphical user interface, including the main menu, game board, tile rendering, score display, and tile animations.

No external dataset or API was required. We also did not reuse an external 2048 implementation for the core game environment or AI agents. The project architecture, game environment and mechanics, Expectimax implementation, N-tuple network, TD-learning pipeline, and UI were developed by our group.

Our project does not use an external dataset. Instead, the reinforcement learning training data is generated automatically through self-play.

The project can be run locally after installing the dependencies listed in `requirements.txt`.